

大数据技术与基础

课后作业

学号：2106410121

姓名：季文龙

班级：计算机 2101

抓取一个公开网站上的数据，数据必须是多页，存为 csv 格式；如果数据是多个 pdf，将其合并为一个 pdf。程序要尽量自动化，减少人为干预。

代码：

```
import requests
import pandas as pd
import json
import time

pd.options.display.max_rows=500
headers = {'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:73.0) Gecko/20100101 Firefox/73.0'}

url = 'https://c.m.163.com/ug/api/wuhan/app/data/list-total' # 定义要访问的地址
r = requests.get(url, headers=headers) # 使用 requests 发起请求

data_json = json.loads(r.text)
data = data_json['data']
data_province = data['areaTree'][2]['children']
areaTree = data['areaTree']

class spider_yiqing(object):

    # 将提取数据的方法封装成函数
    def get_data(data, info_list):
        info = pd.DataFrame(data)[info_list] # 主要信息

        today_data = pd.DataFrame([i['today'] for i in data]) # 提取 today 的数据
        today_data.columns = ['today_' + i for i in today_data.columns]

        total_data = pd.DataFrame([i['total'] for i in data])
        total_data.columns = ['total_' + i for i in total_data.columns]

        return pd.concat([info, total_data, today_data], axis=1)

    def save_data(data,name):
        file_name =
name+'_'+time.strftime('%Y_%m_%d',time.localtime(time.time()))+'.csv'
        data.to_csv(file_name,index=None,encoding='utf_8_sig')
        print(file_name+'保存成功! ')

if __name__ == '__main__':
```

```

today_province = get_data(data_province, ['id', 'lastUpdateTime', 'name'])
today_world = get_data(areaTree, ['id', 'lastUpdateTime', 'name'])
save_data(today_province, 'today_province')
save_data(today_world, 'today_world')

```

程序说明：

此脚本使用请求库向指定的 URL 发出 GET 请求。然后，它从响应中提取数据，并将其保存为 pandas DataFrame。提取的数据包括不同省份和全球新冠肺炎病例的信息。

运行截图：

```

In [1]: runfile('D:/新建文件夹 (4)/未命名3.py', wdir='D:/新建文件夹 (4)')
today_province_2023_10_16.csv保存成功！
today_world_2023_10_16.csv保存成功！

```

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
7	220000	吉林	40485	0	40433	5	0	0	28266	11	16929	0	2	0	-2				
8	110000	北京	28860	0	17514	20	0	0				471	464	0	7				
9	510000	四川	12023	0	10354	5	0	0				104	27	0	77				
10	460000	海南	10436	0	9138	6	0	0				2	14	0	-12				
11	500000	重庆	10181	0	7480	7	0	0				209	45	0	164				
12	410000	河南	9089	0	6880	23	0	0				42	325	0	-283				
13	150000	内蒙古	8394	0	8333	1	0	0				5	13	0	-8				
14	350000	福建	7651	0	6812	1	0	0				0	78	0	-78				
15	330000	浙江	6235	0	4851	1	0	0				37	20	0	17				
16	610000	陕西	5589	0	5111	3	0	0				31	8	0	23				
17	530000	云南	5561	0	4104	2	0	0				157	29	0	128				
18	230000	黑龙江	5493	0	5253	13	0	0				16	11	0	5				
19	140000	山西	5142	0	4339	0	0	0				46	295	0	-249				
20	370000	山东	4797	0	4518	8	0	0				25	104	0	-79				
21	320000	江苏	3808	0	3795	0	0	0				13	5	0	8				
22	710000	辽宁	7066	0	7026	9	0	0				7	0	0	-7				

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	id	lastUpdateName	total_conf	total_sus	total_heal	total_dea	total_sev	total_inp	today_conf	today_sus	today_heal	today_dea	today_sev	today_sto	today_input			
2	9577772	突尼斯	1147477	0	0	29279	0	0	0	0	0	0	0	0	0			
3	9507896	塞浦路斯	2440248	0	2410106	17498	0	0	518	743	5							
4	0	中国	9506995	0	468612	31509	0	28824	46184	5552	78			24325	116			
5	1	日本本土	1879839	0	1735909	18433	0	26881	8445	2								
6	2	泰国	4718908	0	4649509	33505	0	0	0	0	0							
7	3	新加坡	2196690	0	2122239	1710	0	0	710	1140	0							
8	4	韩国	23659055	0	27431824	31790	0	0	88448	74644	46							
9	5	澳大利亚	11077191	0	10869679	16940	0	0	2242	0	0							
10	6	德国	37211937	0	36420100	160768	0	0	0	7300	0							
11	7	美国	102242389	0	99225507	1116085	0	0	7187	51249	2							
12	8	马来西亚	5023519	0	4972691	36831	0	0	609	736	0							
13	9	越南	11524273	0	10610894	43184	0	0	71	41	0							
14	89566	圣巴泰勒米	6	0	6	0	0	0	0	0	0							
15	821	肯尼亚	342470	0	336521	5688	0	0	0	0	0							
16	826	伊朗	7560792	0	7335938	144674	0	0	61	37	1							
17	827	以色列	4756955	0	4733241	11988	0	0	831	941	0							
18	828	毛里求斯	16	0	6	2	0	0	0	0	0							
19	829	黎巴嫩	122055	0	1087587	10743	0	0	0	0	0							
20	833	克罗地亚	1261936	0	1241600	17521	0	0	170	350	6							
21	834	奥地利	5682776	0	5610355	21370	0	0	4679	5264	0							
22	835	瑞士	4365574	0	4292193	14364	0	0	0	3398	0							
23	839	希腊	5348487	0	5472780	34779	0	0	0	6822	0							
24	95800	毛里求斯	41424	0	39694	1037	0	0	0	0	0							
25	845	爱沙尼亚	611210	0	524990	2837	0	0	0	0	0							
26	846	北马其顿	345402	0	335590	9608	0	0	0	0	0							
27	850	白俄罗斯	994037	0	985592	7118	0	0	0	0	0							
28	851	列支敦士登	1286127	0	1286137	9473	0	0	430	71	1							
29	852	阿塞拜疆	825965	0	815395	10006	0	0	72	72	0							

葡萄酒数据集（wine.data）搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 xgBoost3 种分类器模型，比较各种分类器在此数据集上的效果。

【提示】：每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

代码：

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# 读取数据集
data = pd.read_csv('wine.data', header=None)

# 将数据集分为特征和标签
X = data.iloc[:, 1:]
y = data.iloc[:, 0]
from sklearn.preprocessing import LabelEncoder
#XGBoost 要求目标变量必须是从 0 开始的整数，而 y 变量取值是 [1, 2, 3],使用 LabelEncoder 进行转换
le = LabelEncoder()
xgy = le.fit_transform(y)

# 将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy, test_size=0.3,
random_state=42)
# 建立决策树模型
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)

# 建立随机森林模型
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)

# 建立 xgBoost 模型

xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)

# 输出各个模型在测试集上的表现
print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)
```

程序说明：

从 wine.data 中读取数据将数据集中除第一列外的所有列赋值给变量 X，将数据集中第一列赋值给变量 y。使用 LabelEncoder 进行对 y 的值进行转换，将数据集分为训练集和测试集，test_size 为 0.3,random_state 为 42,分别建立三种模型，并对模型的性能进行测试，输出各个模型在测试集上的表现，发现随机森林的模型最好

运行截图

```
In [1]: runfile('D:/python/作业/新建文件夹/wine/5.py', wdir='D:/python/作业/新建文件夹/wine')
Decision Tree Classifier score: 0.9629629629629629
Random Forest Classifier score: 1.0
XGBoost Classifier score: 0.9629629629629629

This version of python seems to be incorrectly compiled
(internal generated filenames are not absolute).
This may make the debugger miss breakpoints.
Related bug: http://bugs.python.org/issue1666807

In [2]:
```